

PROGRAMACIÓN II
TRABAJO PRÁCTICO
ENTREGA FINAL

Integrantes del grupo: Lizarraga Jorge

Docentes: Martha Semken – Gabriel Simoni

Comisión: 02

Cuatrimestre - Año: 2do cuatrimestre 2025

REQUERIMIENTOS DEL SISTEMA

El sistema **HomeSolution** está diseñado para administrar proyectos de mantenimiento integral de viviendas. Cada vez que un cliente contrata un servicio, se crea un proyecto identificado por un número único, que registra información relevante como los datos del cliente (nombre, teléfono, email), dirección de la vivienda, las tareas solicitadas, fechas de inicio y estimada de finalización, y fecha real de finalización. Inicialmente, la fecha estimada y la real coinciden, pero esta última puede modificarse si ocurren retrasos en las tareas.

Cada **tarea** cuenta con un título, descripción, cantidad de días necesarios para realizarla (permitiendo medios días, pero considerando al menos un día completo para estimación) y un empleado responsable. El sistema permite calcular el **costo del proyecto** en base a las horas trabajadas por tarea, aplicando un 35% adicional sobre el total y reduciendo al 25% si hubo retrasos.

La empresa distingue entre **empleados contratados** y **empleados de planta permanente**. Los empleados de planta tienen un valor por día y una categoría ("INICIAL", "TÉCNICO" o "EXPERTO"), y pueden recibir un **adicional del 2%** por tareas sin retrasos. La asignación de empleados puede realizarse de dos formas: el primero libre disponible o priorizando empleados sin retrasos, y en caso de no haber ninguno sin retrasos, se eligen los que menos retrasos hayan tenido. El sistema registra un historial de los empleados asignados a cada proyecto y libera al empleado cuando termina su tarea o cuando finaliza el proyecto.

El sistema debe permitir **modificaciones a proyectos en curso**, como agregar nuevas tareas, ajustando las fechas estimadas y reales de finalización. Además, se registran los retrasos de los empleados y su frecuencia, y se mantiene información de todos los empleados de la empresa. En caso de no haber empleados disponibles para asignar, se lanza una excepción y el proyecto queda como pendiente.

Se consideran también funcionalidades adicionales: obtener tareas sin asignar, consultar el domicilio de un proyecto por número, verificar si un empleado tiene retrasos y obtener la lista completa de empleados. El diseño se realiza pensando en **futuras expansiones**, como la incorporación de nuevas clases de empleados, sin impactar significativamente la estructura del sistema.

Informe Técnico de Clases – Sistema HomeSolution

Clase: Cliente

Tipo: Pública

Interfaz implementada: Icliente

Atributos:

- private String nombre
- private String email
- private String telefono

Constructor:

- public Cliente(String nombre, String email, String telefono)

Métodos:

- public void modificarNombre(String nombre)
- public String verNombre()
- public void modificarTelefono(String telefono)
- public String verTelefono()
- public void modificarEmail(String email)
- public String verEmail()

IREP:

- nombre no sea nulo ni vacío.
- email no sea nulo, vacío y contenga el carácter '@'.
- telefono no sea nulo ni vacío.

Clase: Empleado

Tipo: Abstracta

Interfaz implementada: IEmpleado

Atributos:

- public String nombre
- public int legajo
- public static int contadorLegajo
- public boolean disponible
- public boolean retrasoEnTarea
- public int contadorRetrasos

Constructor:

- public Empleado(String nombre)

Métodos:

- public boolean estaDisponible()
- public void modificarDisponible(boolean disponibilidad)
- public String mostrarNombre()
- public void modificarNombre(String nombre)
- public int mostrarLegajo()
- public boolean estaRetrasado()
- public void retrasarTarea()
- public int mostrarCantidadRetrasos()
- public void aumentarRetrasos()

IREP:

- nombre no sea nulo ni vacío.
- legajo sea mayor o igual que 1.
- contadorLegajo sea positivo y secuencial.
- contadorRetrasos sea mayor o igual que 0.
- disponible refleje correctamente el estado del empleado.

Clase: EmpleadoContratado

Tipo: Pública

Superclase: Empleado

Interfaz implementada: IEmpleadoContratado

Atributos:

- private double costoHora

Constructor:

- public EmpleadoContratado(String nombre, double costoHora)

Métodos:

- public double verCostoHora()
- public void asignarCostoHora(double costoHora)
- public double calculoCostoEmpleado(double cantidadDiasFinalizacion)
- public String toString()

IREP:

- nombre no sea nulo ni vacío.
 - costoHora sea mayor que 0.
 - contadorRetrasos sea mayor o igual que 0.
-

Clase: EmpleadoPermanente

Tipo: Pública

Superclase: Empleado

Interfaz implementada: IEmpleadoPermanente

Atributos:

- private double valorDiaTrabajo
- private String categoria
- private double adicional

Constructor:

- `public EmpleadoPermanente(String nombre, double valorDiaTrabajo, String categoria)`

Métodos:

- `public void modificarValorDia(double nuevoValor)`
- `public double verValorDia()`
- `public String verCategoria()`
- `public double verAdicional()`
- `public double calculoCostoEmpleado(double cantidadDiasFinalizacion)`
- `public String toString()`

IREP:

- nombre no sea nulo ni vacío.
 - valorDiaTrabajo sea mayor que 0.
 - categoria no sea nula ni vacía.
 - adicional sea mayor o igual que 1. (No se agrega esta condición en este proyecto)
-

Clase: Estado

Tipo: Pública

Atributos:

- `public static final String activo`
- `public static final String pendiente`
- `public static final String finalizado`

IREP:

El objeto Estado es inmutable.

Las constantes deben mantener los valores:

- `activo = "ACTIVO"`
- `pendiente = "PENDIENTE"`
- `finalizado = "FINALIZADO"`

Clase: HomeSolution

Tipo: Pública

Interfaz implementada: IHomeSolution

Atributos:

- private LinkedHashMap<Integer, Empleado> empleados
- private LinkedHashMap<Integer, Proyecto> proyectos

Constructor:

- public HomeSolution()

Métodos:

- public void registrarEmpleado(String nombre, double valor) throws
- public void registrarEmpleado(String nombre, double valor, String categoria)

// REGISTRO Y GESTIÓN DE PROYECTOS

- public void registrarProyecto(String[] titulos, String[] descripcion, double[] dias,
String domicilio, String[] cliente, String inicio, String fin)

// ASIGNACIÓN Y GESTIÓN DE TAREAS

- public void asignarResponsableEnTarea(Integer numero, String titulo) ;
- public void asignarResponsableMenosRetraso(Integer numero, String titulo);
- public void registrarRetrasoEnTarea(Integer numero, String titulo, double cantidadDias);
- public void agregarTareaEnProyecto(Integer numero, String titulo, String descripcion, double dias);
- public void finalizarTarea(Integer numero, String titulo) ;
- public void finalizarProyecto(Integer numero, String fin);

// REASIGNACIÓN DE EMPLEADOS

- public void reasignarEmpleadoEnProyecto(Integer numero, Integer legajo, String titulo)
- public void reasignarEmpleadoConMenosRetraso(Integer numero, String titulo)

// CONSULTAS Y REPORTES

- public double costoProyecto(Integer numero);
- public List<Tupla<Integer, String>> proyectosFinalizados();
- public List<Tupla<Integer, String>> proyectosPendientes();
- public List<Tupla<Integer, String>> proyectosActivos();
- public Object[] empleadosNoAsignados();
- public boolean estaFinalizado(Integer numero);
- public int consultarCantidadRetrasosEmpleado(Integer legajo);
- public List<Tupla<Integer, String>> empleadosAsignadosAProyecto(Integer numero);

// NUEVOS REQUERIMIENTOS

- public Object[] tareasProyectoNoAsignadas(Integer numero);
- public Object[] tareasDeUnProyecto(Integer numero);
- public String consultarDomicilioProyecto(Integer numero);
- public boolean tieneRetrasos(Integer legajo);
- public List<Tupla<Integer, String>> empleados();
- public String consultarProyecto(Integer numero);

IREP:

- empleados y proyectos no sean nulos.
- Las claves sean únicas y representen correctamente legajos o IDs.
- Los proyectos tengan fechas válidas (inicio ≤ fin).
- No existan valores negativos en parámetros monetarios o de días.
- No existan empleados duplicados ni proyectos con el mismo ID.

Clase: Proyecto

Tipo: Pública

Interfaz implementada: Iproyecto

Atributos:

- private int idProyecto
- private static int contadorIdProyecto
- private String direccionVivienda
- private LocalDate fechaInicioProyecto

- private LocalDate fechaEstimadaDeFinalizacion
- private LocalDate fechaRealDeFinalizacion
- private double costoProyecto
- private String estado
- private boolean finalizado
- private LinkedHashMap<String, Tarea> tareas
- private Cliente cliente

Constructor:

- public Proyecto(String[] cliente, String[] titulos, String[] descripcion, double[] duracion, String direccionVivienda, String fechaInicioProyecto, String fechaEstimadaFinProyecto)

Métodos:

- public int verId();
- public void agregarTarea(String titulo, String descripcion, double cantidadDiasFinalizacion);
- public void actualizarFinalizado();
- public double calculoCostoFinal();
- public void modificarDireccionVivienda(String nuevaDireccion);
- public String verDireccion();
- public List<Tarea> verTareas();
- public String verEstado();
- public Tarea seleccionarTarea(String titulo);
- public void modificarTarea(String titulo, Tarea tarea);
- public void eliminarTarea(String titulo);
- public void actualizarFechaRealFinalizacion(LocalDate fecha);
- public void actualizarFechaEstimadaFinalizacion(LocalDate fecha);
- public LocalDate verFechaRealFinalizacion();
- public LocalDate verFechaEstimadaFinalizacion();
- public LocalDate verFechaDelInicio();
- public boolean estaFinalizado();
- public void cambiarEstado(String estado);

IREP:

- idProyecto sea mayor o igual que 1.
- cliente no sea nulo.
- tareas no sea nula ni contenga títulos repetidos.
- Las fechas sean válidas ($\text{inicio} \leq \text{estimada} \leq \text{real}$).
- estado sea uno de los definidos en Estado.

Clase: Tarea

Tipo: Pública

Interfaz implementada: ITarea

Atributos:

- private String titulo
- private String descripcion
- private double cantidadDiasFinalizacion
- private double costo
- private Empleado empleado
- private boolean tieneRetraso
- private boolean finalizada

Constructor:

- public Tarea(String titulo, String descripcion, double cantidadDiasFinalizacion)

Métodos:

- public void asignarEmpleado(Empleado empleado);
- public String verTitulo();
- public void modificarTitulo(String nuevoTitulo);
- public String verDescripcion();
- public void modificarDescripcion(String nuevaDescripcion);
- public double VerCantidadDiasFinalizacion();
- public void modificarCantidadDiasFinalizacion(double dias);
- public Empleado verEmpleado();
- public void retrasarTarea(int horas);
- public void retrasarTarea();
- public void calcularCosto();
- public double verCosto();
- public void finalizarTarea();
- public boolean tieneRetrasos();
- public boolean estaFinalizada();

IREP:

- cantidadDiasFinalizacion sea mayor que 0.
 - costo sea mayor o igual que 0. (sigue la condición de que los empleados tengan un valor >0).
 - empleado pueda ser nulo solo si la tarea no está asignada.
 - Si finalizada es true, empleado debe estar disponible.
-

Clase: Tupla

Tipo: Pública y Genérica

Parámetros de tipo: <T1, T2>

Atributos:

- private T1 valor1
- private T2 valor2

Constructor:

- public Tupla(T1 valor1, T2 valor2)

Métodos:

- public T1 getValor1()
- public void setValor1(T1 valor1)
- public T2 getValor2()
- public void setValor2(T2 valor2)



Uso de Conceptos de Programación Orientada a Objetos

Herencia : Se utiliza herencia en las clases EmpleadoContratado y EmpleadoPermanente para que utilicen las variables, metodos y constructor de la clase Padre Empleado.

Polimorfismo: La clase Empleado sirve como clase base y permite definir un comportamiento común para todos los tipos de empleados. Gracias al polimorfismo, se pueden crear distintas subclases como EmpleadoContratado y EmpleadoPermanente, que heredan de Empleado y pueden implementar o extender su comportamiento específico. Esto permite tratar a todos los empleados de manera uniforme a través de referencias de tipo Empleado, aunque cada uno tenga diferencias en su implementación.

Sobreescritura y StringBuilder: La sobreescritura es utilizada en los metodos ToString y en cada una de las implementaciones de las Interfaces, tambien se sobreescriben los metodos de calculo de costo de las clases hijas de Empleado. Cada clase utiliza el mismo metodo con el mismo parametro pero su implementación interna es totalmente diferente, por lo que podemos afirmar que se utiliza el polimorfismo de forma correcta.

Abstracción: Se logra mediante la clase abstracta (Empleado) e interfaces, que definen qué hace un objeto, pero no cómo lo hace.

```
@Override
public String toString() {
    String fin;
    String retraso;
    StringBuilder stringBuilder = new StringBuilder();
    stringBuilder.append("=====Detalle de Proyecto=====").append('\n');
    stringBuilder.append("idProyecto: ").append(idProyecto).append('\n');
    stringBuilder.append("direccion de la vivienda: ").append(direccionVivienda).append('\n');
    stringBuilder.append("fecha de inicio de proyecto: ").append(fechaInicioProyecto).append('\n');
    stringBuilder.append("fecha Estimada de Fin del Proyecto: ").append(fechaEstimadaDeFinalizacion).append('\n');
    stringBuilder.append("fecha real de finalización: ").append(fechaRealDeFinalizacion).append('\n');
    stringBuilder.append("estado del proyecto: ").append(estado).append('\n');
    if(finalizado){
        fin = "Si";
    }
    else{
        fin = "No";
    }
    stringBuilder.append("esta finalizado: ").append(fin).append('\n');
    stringBuilder.append("costo final: ").append(costoproyecto).append('\n');
    if(fechaRealDeFinalizacion.isAfter(fechaEstimadaDeFinalizacion)){
        retraso = "Si";
    }
    else{
        retraso = "No";
    }
    stringBuilder.append("Retraso: ").append(retraso).append('\n');
    stringBuilder.append("=====Detalle de Cliente=====").append('\n');
    stringBuilder.append("cliente: ").append(cliente).append('\n');
    stringBuilder.append("=====Detalle de Tarea=====").append('\n');
    stringBuilder.append("tareass: ").append(tareas).append('\n');
    return stringBuilder.toString();
}
```

Sobrecarga : El concepto de sobrecarga se utiliza a la hora de registrarEmpleados, ya que hay dos metodos con el mismo nombre pero con distinto tipos de parametros.

```
@Override
public void registrarEmpleado(String nombre, double valor) throws
IllegalArgumentException {
    if(valor < 0){
        throw new IllegalArgumentException("El valor no puede ser negativo");
    }
    Empleado empleado = new EmpleadoContratado(nombre,valor);
    empleados.put(empleado.mostrarLegajo(),empleado);
}

@Override
public void registrarEmpleado(String nombre, double valor, String categoria) throws
IllegalArgumentException {
    if(valor < 0){
        throw new IllegalArgumentException("El valor no puede ser negativo");
    }

    List<String> categoriasValidas = List.of("INICIAL", "TECNICO", "EXPERTO");
    if (!categoriasValidas.contains(categoria)) {
        throw new IllegalArgumentException("Ingrese una categoria valida");
    }

    Empleado empleado = new EmpleadoPermanente(nombre, valor, categoria);
    empleados.put(empleado.mostrarLegajo(), empleado);
}
```

Interfaces : Todas las clases contienen sus propias interfaces.

Clases Abstractas : Se utiliza la clase abstracta Empleado, a tal fin que sea solo utilizada por sus hijos EmpleadoContratado y EmpleadoPermanente, De esta forma los hijos toman los atributos, constructor y metodos de la subclase.

Iteradores y foreach :

```
@Override
public Object[] empleadosNoAsignados() {
    List<Object> lista = new ArrayList<>();
    Iterator<Empleado> it = empleados.values().iterator();

    while (it.hasNext()) {
        Empleado empleado = it.next();
        if (empleado.estaDisponible()) {
            lista.add(empleado);
        }
    }
    return lista.toArray();
}

@Override
public List<Tupla<Integer, String>> proyectosActivos() {
    List<Tupla<Integer, String>> listaTupla = new ArrayList<>();
    for (Proyecto proyecto : proyectos.values()){
        if(proyecto.verEstado().equals(Estado.activo)){
            listaTupla.add(new Tupla<Integer, String>(proyecto.verId(),
proyecto.verDireccion()));
        }
    }
    return listaTupla;
}
```

Explicar la complejidad lograda y justificar por medio de Álgebra de Órdenes para el punto:

- 9. Reasignar empleados de ser necesario (se debe poder cambiar un empleado asignado por otro) en $O(1)$
- 10. Reasignar empleado de forma eficiente (buscar el que menos atrasos tenga)

```
@Override
public void reasignarEmpleadoEnProyecto(Integer numero, Integer legajo,
String titulo) throws Exception {
    Proyecto proyecto = proyectos.get(numero);    1
    if(proyecto != null){    1
        Tarea tarea = proyecto.seleccionarTarea(titulo);    1
        Empleado empleado = empleados.get(legajo);    1
        if(tarea != null){    1
            if(tarea.verEmpleado() == null){    //No es peor caso
                throw new Exception("No se encuentra empleado asignado
anteriormente");
            }
            if(empleado != null && empleado.estaDisponible()){    2
                tarea.verEmpleado().modificarDisponible(true);    1
                tarea.asignarEmpleado(empleado);    1
                empleado.modificarDisponible(false);    1
            }
        }
    }
}

F(n) = 1+1+1+1+1+1+2+1+1+1
F(n) = o(10) => aplico regla 4
F(n) = o(1)
```

reasignarEmpleadoEnProyecto tiene un Orden de complejidad de $O(1)$, ya que no se utiliza iteradores para encontrar objetos gracias a la estructura LinkedHashMap que me permite encontrar un valor con solo ingresar una clave.


```

@Override
public void reasignarEmpleadoConMenosRetraso(Integer numero, String titulo) throws
Exception {
    Empleado empleadoConMenorRetraso = null;      1
    boolean proyectoEncontrado = false;          1

    Proyecto proyecto = proyectos.get(numero);    1
    if(proyecto != null){                          1
        proyectoEncontrado = true;                1
        if (proyecto.verEstado().equals(Estado.finalizado)) { //No es peor caso
            throw new Exception("El proyecto se encuentra finalizado");
        }
    }

    if (!proyectoEncontrado) { //No es peor caso
        throw new Exception("Proyecto no encontrado");
    }

    for (Empleado empleado : empleados.values()) { n*(1+3+1)
        if (empleado.estaDisponible()) {
            if (empleadoConMenorRetraso == null || empleado.mostrarCantidadRetrasos()
< empleadoConMenorRetraso.mostrarCantidadRetrasos()) {
                empleadoConMenorRetraso = empleado;
            }
        }
    }

    if (empleadoConMenorRetraso == null) { //No es peor caso
        throw new Exception("No se encuentra empleado disponible");
    }
    Tarea tarea = proyecto.seleccionarTarea(titulo);      1
    if(tarea != null){                                     1
        tarea.asignarEmpleado(empleadoConMenorRetraso);    1
        empleadoConMenorRetraso.modificarDisponible(false); 1
    }
}

```

$F(n) = 1+1+1+1+1+n(1+3+1)+1+1+1+1$

$F(n) = O(9)+O(5n) \Rightarrow$ aplico regla 3

$F(n) = O(9)+O(5) * O(n)$

$F(n) = O(14) * O(n) \Rightarrow$ aplico regla 1

$F(n) = O(1) * O(n)$

$F(n) = o(n)$

reasignarEmpleadoConMenosRetraso tiene una complejidad de $O(n)$

Conclusión

A lo largo del desarrollo de este proyecto pude comprobar en la práctica la importancia de un buen diseño y de la correcta elección de estructuras de datos para lograr sistemas más eficientes y fáciles de mantener. El proceso me permitió aplicar de manera integral los pilares fundamentales de la programación orientada a objetos: encapsulamiento, abstracción, herencia y polimorfismo.

Además, utilicé conceptos como sobreescritura, sobrecarga, interfaces y relaciones entre clases, lo que reforzó mi comprensión de cómo cada herramienta del paradigma aporta una solución adecuada según el contexto.

La elaboración del modelo y su posterior implementación evidenciaron que un diseño claro, coherente y modular simplifica el desarrollo, reduce la complejidad y permite que el sistema sea más escalable. Este trabajo no solo consolidó los contenidos vistos durante la cursada, sino que también me permitió entender cómo se integran todas las piezas para formar un software funcional, robusto y mantenible.